BattleStatistics
- health: double
+ DEFAULT_PLAYER_DAMAGE_REDUCER: double
- magnifier: double
- attack: double
- defence: double
- invincible: boolean
+ DEFAULT_ENEMY_DAMAGE_REDUCER: double
- enabled: boolean
+ DEFAULT_DAMAGE_MAGNIFIER: double
- reducer: double
+ BattleStatistics(double, double, double, double, double)
+ BattleStatistics(double, double, double, double, double, boolean, boolean)
+ battle(BattleStatistics, BattleStatistics): List<BattleRound>
+ getMagnifier(): double
+ getReducer(): double
+ setHealth(double): void
+ isEnabled(): boolean
+ setMagnifier(double): void
+ setInvincible(boolean): void
+ setEnabled(boolean): void
+ setDefence(double): void
+ getDefence(): double
+ isInvincible(): boolean
+ getAttack(): double
+ getHealth(): double
+ applyBuff(BattleStatistics, BattleStatistics): BattleStatistics
+ setReducer(double): void
+ setAttack(double): void
BattleRound
- deltaSelfHealth: double
- deltaTargetHealth: double
+ BattleRound(double, double)
InvincibleState
+ InvisibleState(Player)
+ transitionInvincible(): void
+ transitionInvisible(): void
+ transitionBase(): void
InvisibleState
+ InvisibleState(Player)
+ transitionInvincible(): void
+ transitionInvisible(): void
+ transitionBase(): void
BaseState
+ BaseState(Player)
+ transitionInvincible(): void
+ transitionInvisible(): void
+ transitionBase(): void
PlayerState
- isInvisible: boolean
- player: Player
- isInvincible: boolean
~ PlayerState(Player, boolean, boolean):
+ getPlayer(): Player
+ isInvincible(): boolean
+ transitionBase(): void
+ transitionInvincible(): void
+ transitionInvisible(): void
+ isInvisible(): boolean
Player
+ DEFAULT_ATTACK: double
+ DEFAULT_HEALTH: double
- battleStatistics: BattleStatistics
- inventory: Inventory
- queue: Queue<Potion>
- inEffective: Potion
- nextTrigger: int
- potionListeners: Set<PotionListener>
- collectedTreasureCount: int
- state: PlayerState
+ Player(Position, double, double)
+ getCollectedTreasureCount(): int
+ hasWeapon(): boolean
+ getWeapon(): Useable
+ getBuildables(): List<String>
+ build(String, EntityFactory): boolean
+ move(GameMap, Direction): void
+ onOverlap(GameMap, Entity): void
+ canMoveOnto(GameMap, Entity): boolean
+ getEntity(String): Entity
+ pickUp(Entity): boolean
+ getInventory(): Inventory
+ getEffectivePotion(): Potion
+ <T extends InventoryItem> use(Class<T>): void
+ use(Bomb, GameMap): void
+ triggerNext(int): void
+ changeState(PlayerState): void
+ use(Potion, int): void
+ onTick(int): void
+ remove(InventoryItem): void
+ getBattleStatistics(): BattleStatistics
+ <T extends InventoryItem> countEntityOfType(Class<T>): int
+ applyBuff(BattleStatistics): BattleStatistics
+ onMovedAway(GameMap, Entity): void
+ onDestroy(GameMap): void
+ registerPotionListener(PotionListener): void
+ removePotionListener(PotionListener): void
<<Interface>>
PotionListener
+ notifyPotion(Potion): void
+ notifyNoPotion(): void
ZombieToast
+ DEFAULT_ATTACK: double
- randGen: Random
- movementType: String
+ DEFAULT_HEALTH: double
+ ZombieToast(Position, double, double)
+ notifyPotion(Potion): void
+ notifyNoPotion(): void
+ move(Game): void
GameBuilder
- configName: String
- dungeonName: String
- config: JSONObject
- dungeon: JSONObject
+ buildGame(): Game
+ loadConfig(): void
+ loadDungeon(): void
+ buildMap(Game): void
+ buildGoals(Game): void
DungeonManiaController
- game: Game
+ DungeonManiaController()
+ getSkin(): String
+ rewind(int): DungeonResponse
+ getDungeonResponseModel(): DungeonResponse
+ newGame(String, String): DungeonResponse
+ interact(String): DungeonResponse
+ tick(String): DungeonResponse
+ dungeons(): List<String>
+ tick(Direction): DungeonResponse
+ getLocalisation(): String
+ build(String): DungeonResponse
+ configs(): List<String>
+ generateDungeon(int, int, int, int, String): DungeonResponse
<<Interface>>
Battleable
+ getBattleStatistics(): BattleStatistics
Enemy
- battleStatistics: BattleStatistics
+ Enemy(Position, double, double)
+ onOverlap(GameMap, Entity): void
+ move(Game): void
+ getBattleStatistics(): BattleStatistics
+ onDestroy(GameMap): void
+ onMovedAway(GameMap, Entity): void
+ canMoveOnto(GameMap, Entity): boolean
Mercenary
+ DEFAULT_BRIBE_RADIUS: int
+ DEFAULT_HEALTH: double
- isAdjacentToPlayer: boolean
- bribeAmount: int
- allied: boolean
- allyAttack: double
- allyDefence: double
- movementType: String
+ DEFAULT_BRIBE_AMOUNT: int
+ DEFAULT_ATTACK: double
- bribeRadius: int
+ Mercenary(Position, double, double, int, int, double, double)
- bribe(Player): void
+ notifyNoPotion(): void
+ isAllied(): boolean
- canBeBribed(Player): boolean
+ notifyPotion(Potion): void
+ move(Game): void
+ getBattleStatistics(): BattleStatistics
+ onOverlap(GameMap, Entity): void
+ interact(Player, Game): void
+ isInteractable(Player): boolean
EntityFactory
- config: JSONObject
- ranGen: Random
+ EntityFactory(JSONObject)
+ createEntity(JSONObject): Entity
+ spawnSpider(Game): void
+ spawnZombie(Game, ZombieToastSpawner): void
+ buildSpider(Position): Spider
+ buildPlayer(Position): Player
+ buildZombieToast(Position): ZombieToast
+ buildZombieToastSpawner(Position): ZombieToastSpawner
+ buildMercenary(Position): Mercenary
+ buildBow(): Bow
+ buildShield(): Shield
+ constructEntity(JSONObject, JSONObject): Entity
Game
- id: String
- name: String
- goals: Goal
- map: GameMap
- player: Player
- battleFacade: BattleFacade
- entityFactory: EntityFactory
- isInTick: boolean
+ PLAYER_MOVEMENT: int
+ PLAYER_MOVEMENT_CALLBACK: int
+ AI_MOVEMENT: int
+ AI_MOVEMENT_CALLBACK: int
+ ITEM_LONGEVITY_UPDATE: int
- currentAction: ComparableCallback
- tickCount: int
- sub: PriorityQueue<ComparableCallback>
- addingSub: PriorityQueue<ComparableCallback>
+ Game(String)
+ init(): void
+ tick(Direction): Game
+ tick(String): Game
+ battle(Player, Enemy): void
+ build(String): Game
+ interact(String): Game
+ register(Runnable, int, String): void
+ registerOnce(Runnable, int, String): void
+ unsubscribe(String): void
+ tick(): int
+ getTick(): int
+ getId(): String
+ getName(): String
+ getGoals(): Goal
+ setGoals(Goal): void
+ getMap(): GameMap
+ setMap(GameMap): void
+ getEntityFactory(): EntityFactory
+ setEntityFactory(EntityFactory): void
+ getCollectedTreasureCount(): int
+ getPlayer(): Player
+ getBattleFacade(): BattleFacade
GameMap
- game: Game
- nodes: Map<Position, GraphNode>
- player: Player
+ init(): void
- initRegisterBombsAndSwitches(): void
- initPairPortals(): void
- initRegisterMovables(): void
- initRegisterSpawners(): void
- initPotionListeners(): void
+ registerPotionListener(PotionListener): void
+ moveTo(Entity, Position)
+ moveTo(Entity, Directions)
- triggerMovingAwayEvent(Entity)
- triggerOverlapEvent(Entity)
+ canMoveTo(Entity, Position)
+ dijkstraPathFind(Position, Position, Entity): Position
+ removeNode(Entity): void
+ destroyEntitiesOnPosition(int, int): void
+ destroyEntity(Entity): void
+ addEntity(Entity): void
+ addNode(GraphNode): void
+ getEntity(String): Entity
+ getEntities(Position): List<Entity>
+ getEntities(): List<Entity>
+ <T extends Entity> getEntities(Class<T>): List<T>
+ getPlayer(): Player
+ setPlayer(Player): void
+ getGame(): Game
+ setGame(Game): void
GraphNodeFactory
+ createEntity(JSONObject, EntityFactory): GraphNode
- constructEntity(JSONObject, EntityFactory): GraphNode
GraphNode
- weight: int
- entities: List<Entity>
- position: Position
+ GraphNode(Entity, int)
+ GraphNode(Entity, Position, int)
+ GraphNode(Entity)
+ size(): int
+ getWeight(): int
+ addEntity(Entity): void
+ canMoveOnto(GameMap, Entity): boolean
+ getPosition(): Position
+ getEntities(): List<Entity>
+ removeEntity(Entity): void
+ mergeNode(GraphNode): void
EntityResponse
- id: String
- type: String
- position: Position
- isInteractable: boolean
+ EntityResponse (String, String, Position, boolean)
+ isInteractable(): boolean
+ getId(): String
+ getType(): String
+ getPosition(): Position
Position
+ CHARACTER_LAYER: int
+ DOOR_LAYER: int
+ FLOOR_LAYER: int
+ ITEM_LAYER: int
- X: int
- Y: int
- LAYER: int
+ Position(int, int, int)
+ Position(int, int)
+ translateBy(Position, Position): Position
+ isAdjacent(Position, Position): boolean
+ getLayer(): int
+ getY(): int
+ toString(): String
+ getCardinallyAdjacentPositions(): List<Position>
+ calculatePositionBetween(Position, Position): Position
+ getX(): int
+ magnitude(): int
+ translateBy(Position, Direction): Position
+ hashCode(): int
+ asLayer(int): Position
+ translateBy(Position, int, int): Position
+ getAdjacentPositions(): List<Position>
+ equals(Object): boolean
Spider
+ DEFAULT_SPAWN_RATE: int
+ DEFAULT_ATTACK: double
- forward: boolean
+ DEFAULT_HEALTH: double
- movementTrajectory: List<Position>
- nextPositionElement: int
+ Spider(Position, double, double)
- updateNextPosition(): void
+ move(Game): void
<<Interface>>
Interactable
+ interact(Player, Game)
+ isInteractable(Player): boolean
Entity
+ DOOR_LAYER: int
+ CHARACTER_LAYER: int
+ ITEM_LAYER: int
+ FLOOR_LAYER: int
- previousPosition: Position
- position: Position
- facing: Direction
- previousDistinctPosition: Position
- entityId: String
+ Entity(Position)
+ getFacing(): Direction
+ getPosition(): Position
+ setPosition(Position): void
+ canMoveOnto(GameMap, Entity): boolean
+ getPreviousDistinctPosition(): Position
+ onDestroy(GameMap): void
+ translate(Direction): void
+ getPreviousPosition(): Position
+ onOverlap(GameMap, Entity): void
+ onMovedAway(GameMap, Entity): void
+ translate(Position): void
+ setFacing(Direction): void
+ getId(): String
ZombieToastSpawner
- DEFAULT_SPAWN_INTERVAL: int
+ ZombieToastSpawner(Position, int)
+ spawn(Game): void
+ onDestroy(GameMap): void
+ interact(Player, Game): void
+ isInteractable(Player): boolean
+ onOverlap(GameMap, Entity): void
+ onMovedAway(GameMap, Entity): void
Goal
- type: String
- target: int
- goal1: Goal
- goal2: Goal
+ Goal(String)
+ Goal(String, int)
+ Goal(String, Goal, Goal)
+ achieved(Game): boolean
+ toString(Game): String
GoalFactory
+ createGoal(JSONObject, JSONObject):Goal
BattleFacade
- battleResponses: List<BattleResponse>
+ battle(Game, Player, Enemy): void
+ getBattleResponses(): List<BattleResponse>
ComparableCallback
- r: Runnable
- v: int
- entityId: String
- isValid: boolean
- once: boolean
+ ComparableCallback(Runnable, int, String, boolean)
+ ComparableCallback(Runnable, int, String)
+ run(): void
+ equals(Object): boolean
+ compareTo(ComparableCallback): int
+ getId(): String
+ invalidate(): void
+ isValid(): boolean
Exit
+ Exit(Position)
+ canMoveOnto(GameMap, Entity): boolean
+ onOverlap(GameMap, Entity): void
+ onMovedAway(GameMap, Entity): void
+ onDestroy(GameMap): void
Wall
+ Wall(Position)
+ canMoveOnto(GameMap, Entity): boolean
+ onOverlap(GameMap, Entity): void
+ onMovedAway(GameMap, Entity): void
+ onDestroy(GameMap): void
Portal
- color: ColorCodedType
- pair: Portal
+ Portal(Position, ColorCodedType)
+ canMoveOnto(GameMap, Entity): boolean
+ onOverlap(GameMap, Entity): void
+ canTeleportTo(GameMap, Entity): boolean
+ doTeleport(GameMap, Entity): void
+ getDestPositions(GameMap, Entity): List<Position>
+ bind(Portal): void
+ onMovedAway(GameMap, Entity): void
+ onDestroy(GameMap): void
enum ColorCodedType
ResponseBuilder
+ getDungeonResponse(Game): DungeonResponse
- getInventoryResponse(Inventory): List<ItemResponse>
+ getItemResponse(Entity): ItemResponse
+ getEntityResponse(Game, Entity): EntityResponse
+getRoundResponse(BattleRound): RoundResponse
BattleResponse
- battleItems: List<ItemResponse>
- initialEnemyHealth: double
- rounds: List<RoundResponse>
- enemy: String
- initialPlayerHealth: double
+ BattleResponse(String, List<RoundResponse>, List<ItemResponse>, double, double)
+ BattleResponse()
+ getInitialPlayerHealth(): double
+ getBattleItems(): List<ItemResponse>
+ getRounds(): List<RoundResponse>
+ getInitialEnemyHealth(): double
+ getEnemy(): String
RoundResponse
- deltaPlayerHealth: double
- deltaEnemyHealth: double
+ RoundResponse(double, double)
+ getDeltaCharacterHealth(): double
+ getDeltaEnemyHealth(): double
ItemResponse
- id: String
- type: String
+ ItemResponse(String, String)
+ getId(): String
+ getType(): String
Inventory
- items: List<InventoryItem>
+ add(InventoryItem): boolean
+ remove(InventoryItem): void
+ getBuildables(void): List<String>
+ checkBuildCriteria(Player, boolean, boolean, EntityFactory): InventoryItem
+ <T extends InventoryItem> getFirst(Class<T>): T
+ <T extends InventoryItem> count(Class<T>): int
+ getEntity(String): Entity
+ getEntities(): List<Entity>
+ <T> getEntities(Class<T>): List<T>
+ hasWeapon(void): boolean
+ getWeapon(void): Useable
InventoryItem
+ InventoryItem(Position)
+ canMoveOnto(GameMap, Entity): boolean
+ onDestroy(GameMap): void
+ onOverlap(GameMap, Entity): void
+ onMovedAway(GameMap, Entity): void
+ applyBuff(BattleStatistics origin): BattleStatistics
+ getDurability(): int
Boulder
+ Boulder(Position)
+ canMoveOnto(GameMap, Entity): boolean
+ onOverlap(GameMap, Entity): void
+ canPush(GameMap, Direction): boolean
+ onMovedAway(GameMap, Entity): void
+ onDestroy(GameMap): void
Door
- open: boolean
- number: int
+ Door(Position, int)
+ canMoveOnto(GameMap, Entity): boolean
+ onOverlap(GameMap, Entity): void
+ hasKey(Player): boolean
+ isOpen(): boolean
+ open(): void
+ onMovedAway(GameMap, Entity): void
+ onDestroy(GameMap): void
Switch
- activated: boolean
- bombs: List<Bomb>
+ Switch(Position)
+ subscribe(Bomb): void
+ subscribe(Bomb, GameMap): void
+ unsubscribe(Bomb): void
+ canMoveOnto(GameMap, Entity): boolean
+ onOverlap(GameMap, Entity): void
+ activateBombs(GameMap): void
+ onMovedAway(GameMap, Entity): void
+ isActivated(): boolean
+ onDestroy(GameMap): void
Wood
+ Wood(Position)
+ onOverlap(GameMap, Entity): void
+ applyBuff(BattleStatistics origin): BattleStatistics
+ getDurability(): int
Treasure
+ Treasure(Position)
+ onOverlap(GameMap, Entity): void
+ applyBuff(BattleStatistics origin): BattleStatistics
+ getDurability(): int
Key
- number: int
+ Key(Position, int)
+ onOverlap(GameMap, Entity): void
+ applyBuff(BattleStatistics origin): BattleStatistics
+ getDurability(): int
Arrow
+ Arrow(Position)
+ onOverlap(GameMap, Entity): void
+ applyBuff(BattleStatistics origin): BattleStatistics
+ getDurability(): int
Bomb
+ DEFAULT_RADIUS:int
- state: State
- radius: int
- subs: List<Switch>
+ Bomb(Position, int)
+ subscribe(Switch): void
+ onOverlap(GameMap, Entity): void
+ getRadius(): int
+ onPutDown(Gamemap, Position): void
+ getState(): State
+ applyBuff(BattleStatistics origin): BattleStatistics
+ getDurability(): int
Sword
+ DEFAULT_ATTACK: double
+ DEFAULT_ATTACK_SCALE_FACTOR: double
+ DEFAULT_DURABILITY: int
+ DEFAULT_DEFENCE: double
+ DEFAULT_DEFENCE_SCALE_FACTOR: double
- durability: int
- attack: double
+ Sword(Position, double, int)
+ applyBuff(BattleStatistics): BattleStatistics
+ canMoveOnto(GameMap, Entity): boolean
+ onOverlap(GameMap, Entity): void
+ onMovedAway(GameMap, Entity): void
+ onDestroy(GameMap): void
+ use(Game): void
+ getDurability(void): int
Buildable
+ Buildable(Position)
+ onOverlap(GameMap, Entity): void
Potion
- duration:int
+ Potion(Position, int)
+ onOverlap(GameMap, Entity): void
+ getDuration(void): int
+ applyBuff(BattleStatistics): BattleStatistics
+ getDurability(): int
Bow
- durability: int
+ Bow(int)
+ use(Game): void
+ applyBuff(BattleStatistics): BattleStatistics
+ getDurability(void): int
Shield
- durability: int
- defence: double
+ Shield(int, double)
+ use(Game): void
+ applyBuff(BattleStatistics): BattleStatistics
+ getDurability(void): int
InvisibilityPotion
+ DEFAULT_DURATION: int
+ InvisibilityPotion(Position, int)
+ applyBuff(BattleStatistics): BattleStatistics
InvincibilityPotion
+ DEFAULT_DURATION: int
+ InvincibilityPotion(Position, int)
+ applyBuff(BattleStatistics): BattleStatistics
<<Interface>>
Useable
+ use():void